can encrypt something with your server's public key, and only your server can decrypt it. A complete TLS certificate consists of both the public and private parts.

When a browser connects to your app, the server sends the public key part of the TLS certificate. But how does the browser know that it was definitely *your* server that sent the certificate? To achieve this, your TLS certificate contains additional certificates, including one or more certificates from a third party, a certificate authority (CA). At the end of the *certificate chain* is the *root certificate*.

CAs are special trusted entities, and browsers are hardcoded to trust specific root certificates. For the TLS certificate for your app to be trusted, it must contain (or be signed by) a trusted root certificate. Browsers periodically update their internal list of root certificates and revoke root certificates that can no longer be trusted.

When you use the ASP.NET Core development certificate, or if you create your own self-signed certificate, your site's HTTPS is missing that trusted root certificate. That means browsers won't trust your certificate and won't connect to your server by default. To get around this, you need to tell your development machine to explicitly trust the certificate.

In production, you can't use a development or self-signed certificate, as a user's browser won't trust it. Instead, you need to obtain a signed HTTPS certificate from a service like Let's Encrypt or from a cloud provider like AWS, Azure, or Cloudflare. These certificates are already signed by a trusted CA, so they are automatically trusted by browsers.

